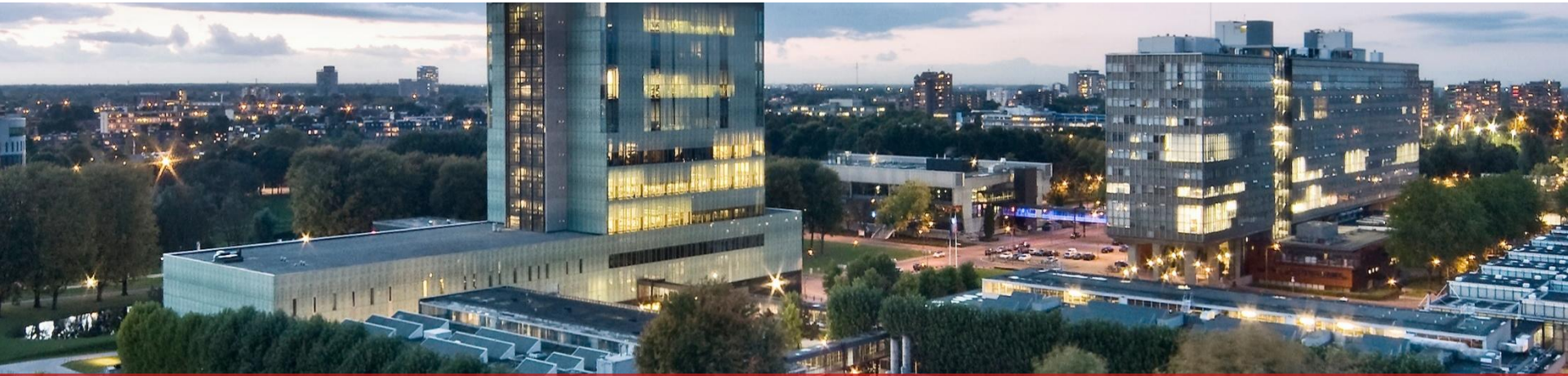
TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY



Domain Specific Language Design (2IMP20)

Code Generation

Loek Cleophas, Ivan Kurtev

Agenda

Code Generation/Model-to-text transformation

- Overview
- Languages and language constructs for code generation
  - Templates

Model-to-Text Transformations

Two common scenarios (among others) for using models:

- model execution (for example by using a model interpreter)
- model transformation: transform models to other models for which we have tools

A particular type of model transformation is *Model-to-Text transformation*

- one or more input models
- result is text:
  - code in a programming language
  - documentation (e.g. in HTML)
  - any other useful form

Model-to-Text Transformations

Model-to-text versus *model-to-model* transformations

Model-to-model:

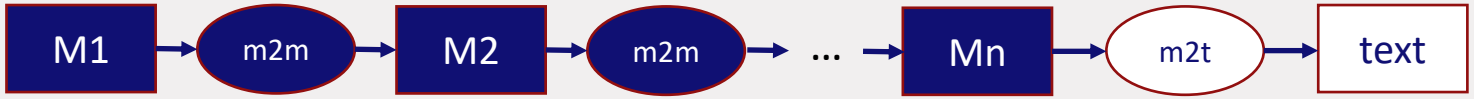
- the result is a model/models, a structure that conforms to a metamodel
- transformation rules map metaelements (from the input and output metamodels), i.e. the *abstract syntax*

Model-to-text:

- the result is text, not necessarily in a formally defined language or adhering to a structure
- transformation rules map source metaelements to text, i.e. the *concrete target syntax*
  - even if the result is in a language with a metamodel, this metamodel is not used directly

Model-to-Text Transformations

Model-to-text transformations are usually the last step in a transformation chain:



If the last model is complete and in a language with textual syntax, the last step is just a *pretty printing*

- often the pretty printer can be automatically derived from the textual syntax definition (for example, Xtext supports this)

In many cases the model-to-text transformation is an implementation of a translation from one language to another

- the transformation may have complex logic
- model-to-model transformation may not be applicable because target metamodel is not available

Examples

Throughout this course:

- Generation of Java code from EMF metamodels
- Generation of ANTLR grammar, parser and editors from Xtext specification
- …

Many others:

- Code in a GPL from UML class diagrams and state machines
- Database schemas from ER models

Model-to-Text Transformation Languages

Similarly to model-to-model transformations, model-to-text transformation languages have been proposed

- Examples: Epsilon Generation Language (EGL), Acceleo, MofScript, Xtend

These languages are used for code generation where a textual artifact is produced from models:

- typically, the result is expressed in another language (GPL or DSL) with available tools (e.g. compiler, interpreter)

Model-to-Text Transformation Languages

A model-to-text transformation is decomposed into transformation rules

The target part of a rule is called *template* and has two parts:

- *static part* – text included verbatim in the result, called *object code* written in the target *object language*
- *dynamic part* – executable code (*meta code*) that usually performs navigation and extraction from the input models

The dynamic part creates ‘holes’ in the template object code. After executing the dynamic part, these ‘holes’ are filled with object code typically derived from the input models

Model-to-Text Transformation Languages

Example model-to-text transformation rule (in Xtend, explained later):

```
def slcoModel2POOSL(Model model){
    '''
    import "../lib/structures.poosl"

    «FOR cl : model.classes»
    «class2POOSL(cl)»

    «ENDFOR»
    system
    instances
        «FOR o : model.objects»
        «o.name» : «o.class_.name»
        «ENDFOR»
    channels
        «FOR c : model.channels.filter(BidirectionalChannel).filter(it | it.channelType == ChannelTypeEnum::SYNCHRONOUS)»
        {«c.object1.name».«c.port1.name», «c.object2.name».«c.port2.name»}
        «ENDFOR»
    '''
}
```

Model-to-Text Transformation Languages

After executing a model-to-text transformation, the result may be

- *complete*: no need for additional manually written code
- *incomplete*: parts have to be added manually after the generation

The result is incomplete if the source model does not have all the needed information (possibly because the source language is not expressive enough)

- Example: the Java code generated from an Ecore metamodel that contains classes with operations. The methods created from the operations have empty bodies that need to be completed manually

Example Code Generation Task: SLCO to POOSL

SLCO:

- allows specification of models that contain communicating objects
- objects communicate via messages sent over channels
- object behavior is defined in a one or more state machines running in parallel
- every object runs in its own process

We have defined textual syntax for SLCO

Still, we have no tools for executing SLCO programs

Example Code Generation Task: SLCO to POOSL

Executing SLCO programs:

- Translation to program in another language for which we have an execution engine
- The translation will be implemented as a model-to-text transformation written in *Xtend*

Target language:

- POOSL: Parallel Object-Oriented Simulation Language

Xtend:

- Part of the Xtext language workbench
- Java-like programming language with dedicated code generation constructs: template methods and *template expressions*

Example Code Generation Task: SLCO to POOSL

In effect this is an example of a *language translation*

- The model-to-text transformation acts as a *compiler*

The complexity of the translation depends on the degree of similarity between the source and target languages

- The challenge in designing the model-to-text transformation is to decide how the source language constructs are interpreted in terms of the target language constructs

In our case, POOSL and SLCO have a large degree of *similarity*

Aligning SLCO and POOSL

SLCO	POOSL
Program • Classes, objects, channels	System • Process classes, objects, channels
Class • ports, variables and state machines	Process class • ports, message declarations, variables, methods
State machines running in parallel	Parallel execution construct Methods (for implementing SLCO state machines)
Sending and receiving named signals with parameters	Message declarations in a process class Sending and receiving named messages with parameters
Channels • Unidirectional vs. bidirectional • Synchronous, Asynchronous (Lossy and Lossless)	Bidirectional synchronous channels

SLCO primitive types, assignments, delays, variables and expressions have the same counterparts in POOSL

Aligning SLCO and POOSL

Channels:

- Only SLCO models with bidirectional synchronous channels will be translated
- Other channel kinds can also be translated but their semantics need to be implemented in the available POOSL constructs (not done here for simplicity)

State machines:

- Every state becomes a POOSL class method
- Transition statements are translated to POOSL statements in the method body
- Transition traversal is a call to the method corresponding to the target state

Implementing the Transformation from SLCO to POOSL

- The alignment has been performed at the level of language concepts even though no POOSL metamodel has been used
- The transformation will be implemented as a model-to-text transformation: the textual representation of the POOSL constructs will be used directly

Implementation technology: Xtend

- Java-like language, Xtend programs compile to Java (another example of code generation!)
- A transformation is organized in classes and their methods using *template expressions*

Xtend

Xtend template expressions:

- Quoted arbitrary text with metacode: expressions, control statements (IF-THEN-ELSE, FOR loops), method calls
- Smart handling of indentations
- Used in Java-like Xtend methods
- Source model elements passed as method parameters
- Metacode can obtain values from the input model

SLCO to POOSL Generation in Xtend

```
def slcoModel2POOSL(Model model){
    '''
    import "../lib/structures.poosl"

    «FOR cl : model.classes»
    «class2POOSL(cl)»

    «ENDFOR»
    system
    instances
        «FOR o : model.objects»
        «o.name» : «o.class_.name»
        «ENDFOR»
    channels
        «FOR c : model.channels.filter(BidirectionalChannel).filter(it | it.channelType == ChannelTypeEnum::SYNCHRONOUS)»
        {«c.object1.name».«c.port1.name», «c.object2.name».«c.port2.name»}
        «ENDFOR»
    '''
}
```

Static text in the target language

Static text in the target language

Dynamic part: expression to be evaluated; result is inserted in the text

Control structures: IF and FOR

- Static part surrounded by ‘’’
- Metacode surrounded by special symbols << and >>

TU/e

SLCO to POOSL Generation in Xtend

We follow the *syntax directed translation* approach: one method for each SLCO metaelement.

Examples:

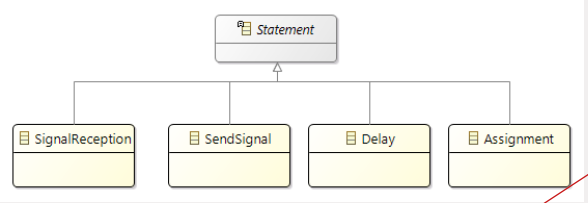
```
def type2POOSL(PrimitiveTypeEnum type){
    switch(type){
        case BOOLEAN : '''Boolean'''
        case INTEGER : '''Integer'''
        case STRING : '''String'''
        default : '''Error: unsupported type'''
    }
}
```

```
def transition2POOSL(Transition t){
    '''
    «FOR s : t.statements»
    «statement2POOSL(s)»
    «ENDFOR»
    «t.target.name»()()
    '''
}
```


TU/e

SLCO to POOSL Generation in Xtend

Dispatch methods for dealing with generalization hierarchies in the metamodel

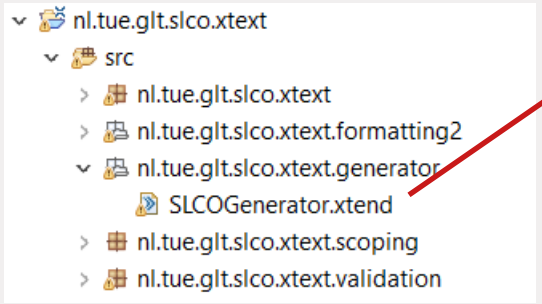



```
def transition2POOSL(Transition t){
    '''
    «FOR s : t.statements»
    «statement2POOSL(s)»
    «ENDFOR»
    «t.target.name»()()
    '''
}
```

```
def dispatch statement2POOSL(Assignment a){
    '''«a.variable.name» := «expression2POOSL(a.expression)»;'''
}

def dispatch statement2POOSL(SignalReception sr){
    '''
    «sr.port.name» ? message(sn, args | sn = "«sr.signalName»");
    «FOR a : sr.arguments»
    «(a as SignalArgumentVariable).variable.name» := args at(«sr.arguments.indexOf(a) + 1»);
    «ENDFOR»
    '''
}

def dispatch statement2POOSL(Delay d){
    '''delay «d.value»;'''
}
```

Depending on the type of the parameter, the correct dispatch method is selected at *runtime* (dynamic dispatch)

Code Generation in the Xtext Workbench



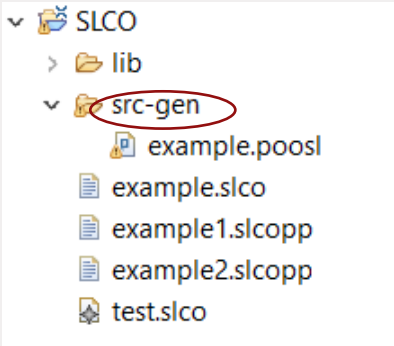

```
/**
 * Generates code from your model files on save.
 *
 * See https://www.eclipse.org/Xtext/documentation/303_runtime_concepts.html#code-generation
 */
class SLCOGenerator extends AbstractGenerator {

    override void doGenerate(Resource resource, IFileSystemAccess2 fsa, IGeneratorContext context) {
        var root = resource.allContents.head as Model

        fsa.generateFile(resource.URI.trimFileExtension.lastSegment + ".poosl", slcoModel2POOSL(root))
    }
```

Xtext automatically generates a *default generator class* for every Xtext-based language project

IFileSystemAccess is a facility for creating files with generated code

- Method *doGenerate* is executed every time the user changes and saves a model
- It has an empty body by default

Code Generation in the Xtext Workbench



- When an Xtext editor is used and a file is saved, the generated code is by default placed under the folder *'src-gen'* in an Eclipse project
- Users can create subfolders if needed

TU/e

Demo: Generation of POOSL from SLCO Programs

- SLCO-to-POOSL generator in Xtend
- Execution of the generated POOSL programs

Code Generation with Xtend: Summary

Xtend is a general purpose programming language, not a DSL for model-to-text transformations

Still, it has been designed for performing code generation tasks

- *template expression* is the key construct
- In the scope of Xtext workbench, Xtend is used for code generation

Users are free to decide on the decomposition and order of the transformation logic

dispatch methods are very handy for defining a syntax-directed code generator (one dispatch method per metaclass)

Other Approaches to Code Generation: EGL

Epsilon Generation Language (EGL, part of the Epsilon platform) is a template-based model-to-text transformation language:

```
[% for (i in Sequence{1..5}) { %]
i is [%= i %]
[% } %]
```

EGL

EGL is a preprocessor for Epsilon Object Language (EOL)

```
[% for (i in Sequence{1..5}) { %]
i is [%= i %]
[% } %]
```

Is translated to (in EOL):

```
for (i in Sequence{1..5}) {
   out.print('i is '); out.println(i);
}
```

Result:

```
i is 1
i is 2
i is 3
i is 4
i is 5
```

Further Discussion on Code Generation

- Protected Regions
- Generation Gap Design Pattern
- Documentation Generation

Manually Changing Generated Code: Protected Regions

Sometimes the generated code is incomplete (or expected to be extended) and a manually written code needs to be inserted in the generated code

- For example, a generator creates method signatures without a body; the developer writes the body manually

If the code is regenerated, the manually written code should be preserved and re-inserted in the same place

Protected regions:

- Designated regions in the generated code that allow to add handwritten code

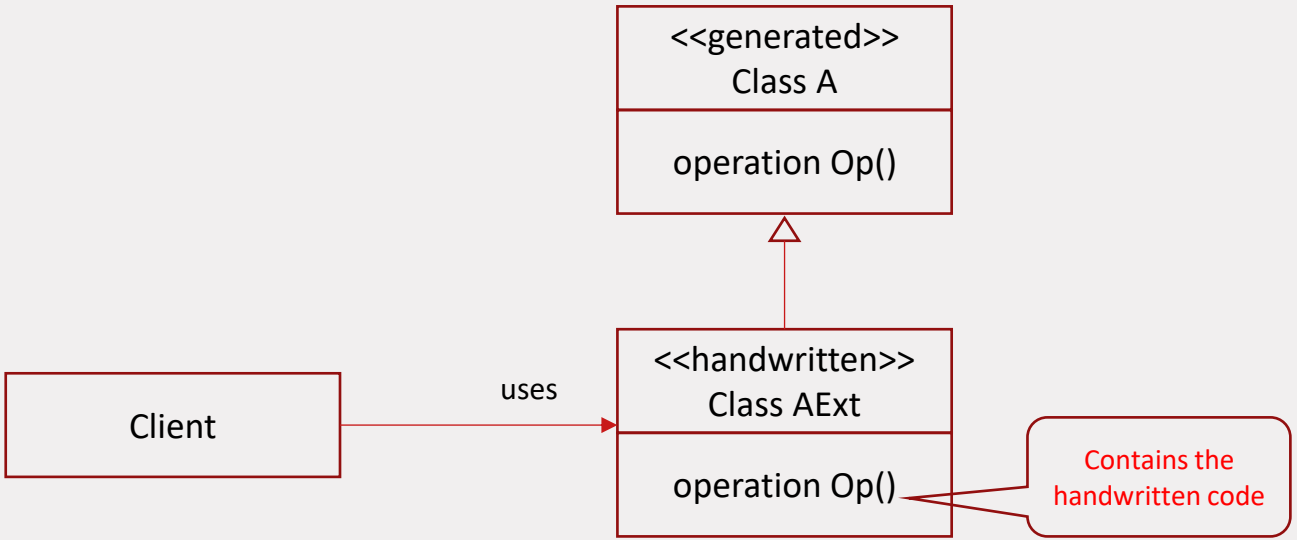
Protected Regions

- Protected regions are usually surrounded by comments and have unique ids
- Model-to-text transformation languages may have support for defining protected regions
  - the processor recognizes the regions and provides options to preserve the text in case of re-generation

- EGL has support for protected regions
- Xtend considered support for protected regions but it was abandoned
  - *Generation Gap pattern* is recommended instead

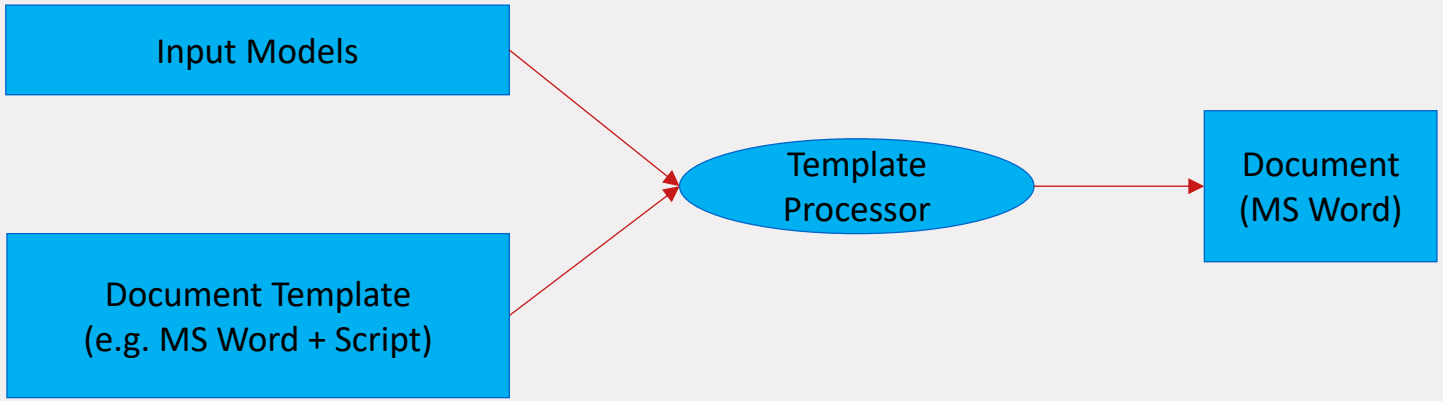
Generation Gap Pattern

- Intention: separates generated from handwritten code
- Relies on inheritance to be available in the target language

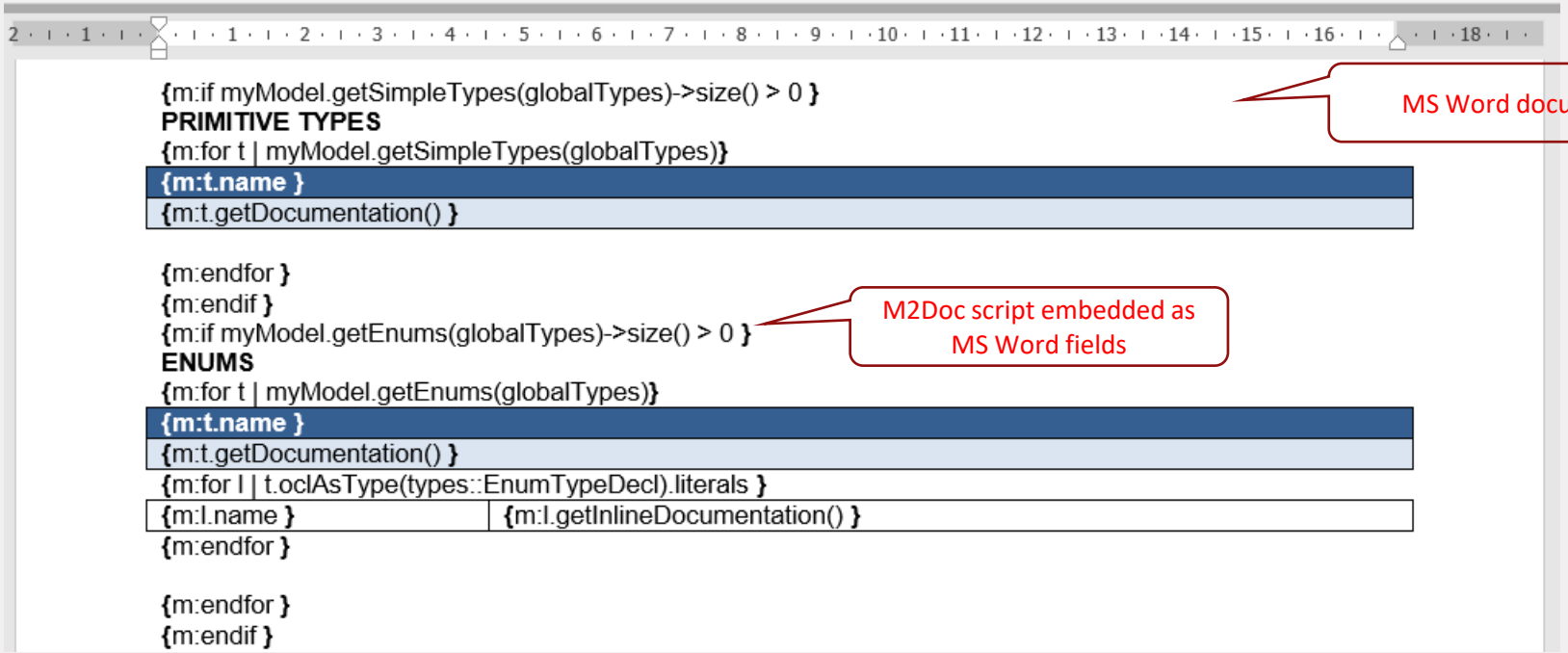


Documentation Generation

- Documentation generation is a special type of transformation in which the result is typically in a document format supported by some text editor, for example, MS Word or OpenOffice
- Metacode script, similar to model-to-text transformations is inserted in an existing document, recognized and executed by a processor
- Some technologies: M2Doc, GenDoc, Birt



Example Word Template and M2Doc Script



Configuring Generators

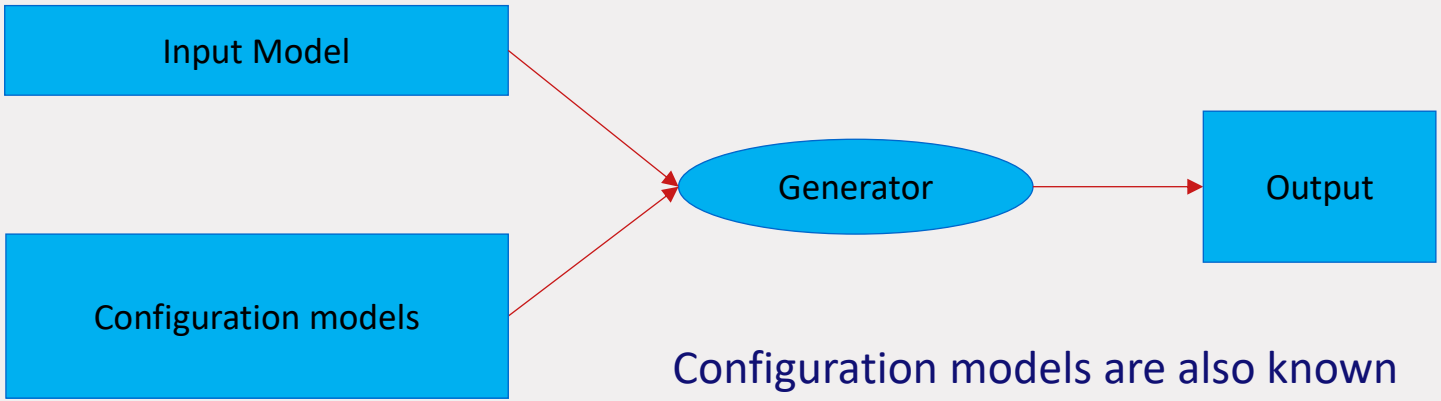
Sometimes a code generator (or a model transformation) has to accept parameters that drive the generation

Examples:

- generator for Java that takes into account the target Java version
- User-defined mappings for DSL primitive types to target language types

Recommended practice is to supply all necessary configuration information in one or more additional input models

Configuring Generators



Configuration models are also known as:

- Generator models
- Decorator models

Translational Semantics

Model transformations combined with code generators allow the translation of models in DSLs to known languages (for which we already have semantics/tools)

- The effect is a pragmatic definition of DSL semantics, known as *translational semantics*

Translational semantics is often used in practice. However, it should not replace the need for formal semantics of a DSL:

- The target language often has no formal semantics
- The model transformation language (or code generator) used to express the translation usually has no formal semantics either

Resources on Code Generation

Code generation with Xtend

- https://www.eclipse.org/xtend/documentation/index.html

EGL

- Epsilon book, Chapter 7